Rapport Technique — GuardianTrack 🛡️

1. Introduction au Projet

GuardianTrack est une application Android de sécurité personnelle conçue pour détecter les chutes et les impacts brutaux subis par l'utilisateur. En cas d'incident, l'application alerte automatiquement un contact d'urgence via SMS (ou notification simulée) et synchronise les données sur un serveur distant via une API REST.

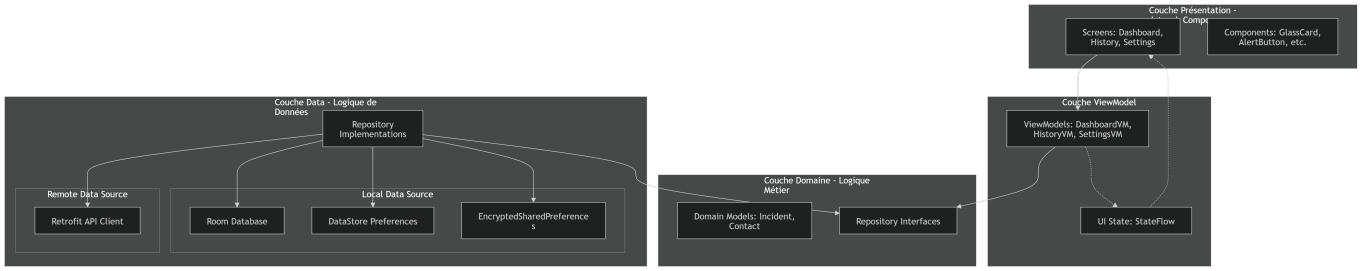
Le projet met l'accent sur la robustesse, la réactivité et la sécurité des données, tout en offrant une interface utilisateur moderne et immersive ("Tactical Dark Glass") entièrement développée avec **Jetpack Compose**.

---

2. Architecture Logicielle (MVVM & Clean Architecture)

GuardianTrack suit le patron d'architecture **MVVM (Model-View-ViewModel)** recommandé par Google, couplé à des principes de **Clean Architecture** pour garantir la séparation des préoccupations et la testabilité.

Diagramme d'Architecture MVVM



Détail des Couches

1. **UI Layer (Compose)** : Gère l'affichage et les interactions utilisateur. Utilise `StateFlow` pour observer les changements d'état et mettre à jour l'interface de manière réactive.
2. **ViewModel Layer** : Agit comme un médiateur entre la vue et le dépôt. Il expose le `StateFlow` consommé par Compose et gère la logique de présentation, tout en survivant aux changements de configuration.
3. **Domain Layer** : Contient les règles métier pures. Les **Use Cases** (optionnels ici mais suggérés par l'architecture) et les modèles de domaine (`Incident`, `Contact`) sont indépendants des détails techniques comme la base de données ou le réseau.
4. **Data Layer** : Responsable de la persistence et de la communication réseau. Elle implémente les interfaces de dépôt définies dans le domaine et gère la logique de synchronisation entre Room et l'API distante.

---

3. Modèle de Données (Room Database)

La persistance locale est assurée par **Room**, une bibliothèque d'abstraction SQLite puissante fournie par Google.

Schéma de la Base de Données

INCIDENTS		
long	id	PK
long	timestamp	
string	type	
double	latitude	
double	longitude	
boolean	is_synced	

EMERGENCY_CONTACTS		
long	_id	PK
string	name	
string	phone_number	

- **Table `incidents`** : Enregistre chaque événement détecté (chute, impact ou manuel). Le champ `is_synced` est crucial pour notre stratégie offline-first, permettant de marquer les données déjà envoyées au serveur.
- **Table `emergency_contacts`** : Stocke les informations des contacts à prévenir. Ces données sont isolées et accessibles via un **ContentProvider** protégé par une signature système.

---

4. Interface Utilisateur (Design System)

L'UI de GuardianTrack repose sur le thème **"Tactical Dark Glass"**, une identité visuelle forte inspirée des instruments de précision aéronautique :

- **Jetpack Compose** : Utilisation exclusive de Compose pour une interface fluide, réactive et modulaire.
- **Glassmorphisme** : Effet de transparence et de flou sur les cartes (`GlassCard`) pour une profondeur visuelle moderne.
- **Animations Avancées** :
  - **Radar pulsant** : Créé via `Canvas` et `rememberInfiniteTransition` pour indiquer l'état de veille active.
  - **Scan line** : Une barre lumineuse qui balaie les cartes actives pour renforcer l'aspect "surveillance".
  - **Réactivité des valeurs** : La magnitude de l'accéléromètre utilise `animateFloatAsState` avec une physique de ressort (`spring`) pour un affichage dynamique et naturel.

---

5. Fonctionnalités Techniques Clés

A. Détection de Chute (Algorithme 2 phases)

L'algorithme de détection est optimisé pour minimiser les faux positifs tout en garantissant une réactivité maximale. Il fonctionne en deux étapes clés basées sur les données brutes de l'accéléromètre :

1. **Phase de Chute Libre** : Détection d'une chute brutale lorsque la magnitude de l'accélération descend en dessous de 3.0 m/s^2 (proche de 0G).
2. **Phase d'Impact** : Détection d'un choc violent immédiatement après la chute libre, avec une magnitude dépassant un seuil paramétrable par l'utilisateur (ex: 15 m/s^2).

Pour garantir la fluidité de l'interface, ces calculs intensifs sont déportés dans un `HandlerThread` dédié, évitant toute saccade sur le thread principal.

B. Services d'arrière-plan et Persistance du Service

- **Foreground Service** : Le `SurveillanceService` affiche une notification persistante obligatoire, garantissant que le système Android ne tue pas le processus de surveillance, même sous pression mémoire.
- **WorkManager & Android 12+** : Utilisation de **Expedited Work** pour relancer automatiquement la surveillance dès le démarrage du téléphone (`BOOT_COMPLETED`), contournant les restrictions strictes d'Android 12 sur le lancement de services en arrière-plan.

C. SMS et Localisation Intelligente

- **Localisation Haute Précision** : Utilisation de `FusedLocationProviderClient` de Google Play Services pour obtenir les coordonnées les plus précises possibles au moment de l'impact.
- **Alerte SMS Contextuelle** : Génération d'un message contenant non seulement l'alerte, mais aussi un lien direct vers Google Maps facilitant l'intervention des secours.
- **Mode Simulation** : Un commutateur dans les réglages (via `DataStore`) permet de tester l'application sans consommer de SMS réels, en affichant simplement des notifications système locales.

---

6. Sécurité et Confidentialité des Données

La sécurité est au cœur de GuardianTrack pour protéger la vie privée de l'utilisateur :

- **EncryptedSharedPreferences** : Les informations sensibles, comme le numéro de téléphone du contact d'urgence, sont chiffrées au repos via l'algorithme AES-256-GCM géré par la bibliothèque Jetpack Security.
- **Permissions Dynamiques** : Mise en place d'un flux de permissions moderne avec des explications pédagogiques avant chaque demande critique (GPS, SMS).
- **ContentProvider Protégé** : L'exposition des contacts d'urgence à d'autres applications est verrouillée par une permission personnalisée avec un niveau de protection `signature|privileged`, limitant l'accès aux seules applications système ou signées par le même développeur.

---

7. Stratégie Offline-First et Synchronisation

GuardianTrack est conçu pour fonctionner dans des conditions réseau difficiles :

1. **Enregistrement Immédiat** : Chaque incident est d'abord stocké localement dans Room.
2. **Tentative d'envoi API** : Une requête `POST` est envoyée via Retrofit vers le serveur MockAPI.
3. **Gestion d'État** : Si l'envoi réussit, l'incident est marqué `is_synced = true`.

4. **Synchronisation Différée** : En cas d'échec (zone blanche, pas de data), **WorkManager** prend le relais. Il planifie une tâche de fond avec la contrainte `NETWORK_CONNECTED`, garantissant que les données seront envoyées dès que le réseau sera disponible, même si l'utilisateur a fermé l'application.

---

8. Réponses aux Questions Techniques

Q1 : Flow vs LiveData : Choix et Justification

Dans ce projet, nous avons privilégié **Kotlin Flow** (et `StateFlow`) par rapport à LiveData pour plusieurs raisons fondamentales :

- **Programmation réactive moderne** : Flow fait partie intégrante des coroutines Kotlin, permettant une intégration native et fluide avec les couches Repository et Room (qui supporte Flow nativement).
- **Opérateurs puissants** : Contrairement à LiveData, Flow offre des opérateurs de transformation riches (`map`, `filter`, `combine`, `flatMapLatest`) bien plus performants pour manipuler des flux de données complexes.
- **Indépendance de la plateforme** : LiveData est lié au cycle de vie Android, ce qui rend les tests unitaires plus complexes. Flow est une bibliothèque pure Kotlin, facilitant le test et la réutilisation du code.
- **Optimisation Compose** : Flow se convertit très naturellement en état Compose via `collectAsStateWithLifecycle()`, garantissant une gestion propre des ressources.

Q2 : Gestion du refus définitif de Localisation

Lorsque l'utilisateur refuse la permission de localisation et coche "Ne plus demander", l'application met en place une stratégie de repli robuste :

- **Dialogue Pédagogique** : Nous affichons un dialogue via le `PermissionManager` expliquant que la détection d'incident perd une grande partie de son utilité sans position GPS.
- **Redirection Paramètres** : Nous proposons un bouton "Paramètres" qui ouvre directement la page d'infos de l'application (`Settings.ACTION_APPLICATION_DETAILS_SETTINGS`) pour permettre une réactivation manuelle.
- **Valeur Sentinelle** : Si l'utilisateur refuse toujours, l'incident est tout de même enregistré avec les coordonnées `0.0 / 0.0`. Cela permet de conserver une trace chronologique de l'incident (heure/date) même sans géolocalisation.

Q3 : Sécurité du ContentProvider

Notre `EmergencyContactProvider` est conçu avec des mesures de sécurité strictes pour empêcher tout accès non autorisé :

- **Permission de Signature** : Le fournisseur est protégé par une permission personnalisée avec un `protectionLevel="signature|privileged"`. Seules les applications signées avec la même clé que GuardianTrack peuvent lire les données.
- **Protection contre l'Injection SQL** : Nous ignorons totalement les clauses `selection` ou `selectionArgs` provenant de l'extérieur. La méthode `query` utilise directement les méthodes Room typées (`getAllContactsSync()`).

- **Isolation des Données** : Nous utilisons `MatrixCursor` pour reconstruire manuellement le curseur retourné, séparant ainsi totalement la structure interne de la base Room de l'interface exposée aux tiers.

Q4 : Android 12+ et Services d'arrière-plan (Boot & Relance)

Depuis Android 12 (API 31), le lancement de services foreground (`startForegroundService`) depuis l'arrière-plan est restreint, notamment après un reboot (`BOOT_COMPLETED`).

- **Solution WorkManager** : Nous utilisons **WorkManager** avec la méthode `setExpedited()`. Android autorise WorkManager à s'exécuter immédiatement même en arrière-plan après un démarrage.
- **BootStartWorker** : C'est ce Worker "Expedited" qui se charge de lancer le `SurveillanceService` de manière sécurisée.
- **Direct Boot** : Nous avons également ajouté le support du mode `directBootAware` pour permettre au service de démarrer dès que le système est prêt, même avant que l'utilisateur n'ait déverrouillé son écran.

Q5 : Séparation Entity / DTO / DomainModel

Pour respecter le principe de responsabilité unique (SRP) et faciliter la maintenance, nous utilisons trois modèles distincts :

- **Entity (Room)** : Structure optimisée pour le stockage local SQLite (ex: clés primaires auto-incrémentées).
- **DTO (Retrofit)** : Structure imposée par l'API distante (ex: format JSON avec des noms de champs spécifiques).
- **DomainModel** : Modèle pur utilisé par l'UI et la logique métier.
- **Avantage concret** : Si le format de l'API change (ex: changement du format de date), nous n'avons qu'à modifier le **DTO** et son **Mapper**, sans jamais impacter nos ViewModels ou nos écrans Compose.

Q6 : WorkManager vs JobScheduler vs AlarmManager

Le choix de **WorkManager** pour GuardianTrack est justifié par sa polyvalence :

- **AlarmManager** : Conçu pour des actions à une heure précise (ex: réveil). Il est trop coûteux en batterie et ne gère pas nativement les contraintes réseau.
- **JobScheduler** : API système complexe dont le comportement varie selon les versions d'Android.
- **WorkManager** : C'est la recommandation moderne de Google. Il choisit intelligemment entre JobScheduler ou AlarmManager selon l'API du téléphone. Il garantit la persistance des tâches (survie au reboot) et permet de définir facilement des contraintes comme `NETWORK_CONNECTED`, ce qui est idéal pour notre synchronisation d'incidents.

---

9. Conclusion

GuardianTrack démontre l’utilisation des technologies Android modernes pour répondre à un besoin critique de sécurité.

- Architecture robuste (MVVM + Clean Architecture)
- Sécurité avancée
- Stratégie offline-first fiable
- Interface moderne avec Jetpack Compose

Le projet respecte les contraintes des versions récentes d'Android (Android 12+) tout en suivant les bonnes pratiques de développement.